Relatório – Trabalho Prático I: Pesquisa Externa

Oct 7, 2026 · @Davi

Universidade Federal de Ouro Preto – UFOP · ICEB · Departamento de Computação – DECOM

BCC203 – Estrutura de Dados II · Prof. Guilherme Tavares de Assis · 2026/2

Integrantes: (preencher nomes e matrículas do grupo)

1. Introdução

1.1 Especificação do problema

O trabalho compara o desempenho de quatro métodos de pesquisa externa implementados em C: (1) acesso sequencial indexado, (2) árvore binária de pesquisa em memória externa, (3) árvore B e (4) árvore B*. Os registros ficam em arquivo binário e têm três campos: `chave` (int), `dado1` (long) e `dado2` (char[5000]), totalizando 5.016 bytes por registro (`registro.h`).

Cada execução mede, separadamente para a criação do índice e para a pesquisa:

- número de transferências (leituras) de itens da memória externa para a interna;
- número de comparações entre chaves de pesquisa;
- tempo de execução (fim menos início).

1.2 Programa

O executável segue a linha de comando exigida:

```
pesquisa <método> <quantidade> <situação> <chave> [-P]
```

- `<método>` : 1 a 4; `<situação>` : 1 (ascendente), 2 (descendente) ou 3 (aleatório).
- `-P` imprime as chaves do arquivo antes da pesquisa, fora de qualquer medição.
- A saída informa se a chave foi encontrada, o registro completo (`dado1` e `dado2`) e as três métricas de criação e de pesquisa.
- No lugar da chave, `-E` ativa o modo de experimento: o índice é criado uma vez e 20 chaves são pesquisadas automaticamente, com saída das médias e de uma linha CSV.
- Argumentos inválidos, método 1 com arquivo não ascendente, falta de memória e erros de arquivo geram mensagens de erro específicas.

O arquivo `registros_<quantidade>_<situação>.bin` é reutilizado se já existir com o tamanho correto; caso contrário, é gerado com as chaves 1..n na ordem pedida (embaralhamento Fisher–Yates com semente fixa 42 na situação 3). A geração fica fora das medições.

1.3 Contagem das métricas

- **Transferências:** cada registro ou nodo lido do disco com `fread` conta 1 (contador global em `metricas.c`).
- **Comparações:** toda comparação entre chaves passa pela macro `COMP(expr)`, que incrementa o contador e avalia a expressão.
- **Tempo:** `clock_gettime(CLOCK_MONOTONIC)`, com resolução de nanossegundos. Impressão de resultados não entra no tempo.

1.4 Testes realizados

- Quantidades: 100, 200, 2.000, 20.000 e 200.000 registros, nas três situações de ordem.
- Método 1 apenas na situação ascendente, pois o índice de páginas exige arquivo ordenado.
- Método 2 nas situações ordenadas apenas até 20.000: com arquivo ordenado a árvore degenera em lista, e 20.000 registros já levaram cerca de 160 s só de construção (crescimento quadrático).
- 2.000.000 de registros: ver seção 1.5.
- Em cada combinação foram pesquisadas 20 chaves distintas, igualmente espaçadas entre 1 e n (todas existentes). As tabelas mostram as médias dessas 20 pesquisas; todas as chaves foram encontradas em todos os testes.
- Ambiente: Apple M4 (10 núcleos), 16 GB de RAM, macOS, gcc com `-O2`. A execução foi automatizada pelo script `experimentos.sh` (modo `-E`).

1.5 Limitações da bateria de testes

Algumas combinações não foram executadas por limite da máquina ou da própria implementação:

Combinação não executada	Motivo
Métodos 3 e 4 com 2.000.000	As árvores guardam os registros completos em RAM. Com 200.000 registros o pico foi de 1,5 a 2,2 GB; com 2.000.000 seriam cerca de 15 a 22 GB, acima dos 16 GB da máquina.

Combinação não executada	Motivo
Método 2, situações 1 e 2, com 200.000 e 2.000.000	A árvore degenera em lista e a construção faz cerca de $n^2/2$ leituras. Com 20.000 levou cerca de 160 s; por extrapolação, 200.000 levaria cerca de 4,5 horas e 2.000.000 cerca de 18 dias.

A função `limparCacheArquivo` usa `posix_fadvise`, que não existe no macOS; por isso, até 200.000 registros os arquivos (no máximo 1 GB) podem ter sido lidos do cache do sistema operacional. Com 2.000.000 (cerca de 10 GB por arquivo) as leituras chegam de fato ao disco, o que explica o salto de tempo nesse tamanho.

2. Método 1 – Acesso sequencial indexado

O método cria o índice mais barato de todos (n/100 leituras, nenhuma comparação) e pesquisa sempre com exatamente 100 transferências, mas suas comparações crescem linearmente com n.

2.1 Implementação

- O arquivo é dividido em páginas de 100 registros (`ITENS_POR_PAGINA`); a tabela de índice guarda, para cada página, seu número e sua primeira chave (até 20.000 páginas, ou seja, 2.000.000 de registros).
- **Criação:** lê o primeiro registro de cada página e salta os outros 99 com `fseek`.
- **Pesquisa:** percorre a tabela sequencialmente até achar a página candidata, lê a página inteira (a última pode estar incompleta) e faz pesquisa sequencial nela.
- Só se aplica ao arquivo ascendente; nas situações 2 e 3 o programa recusa a execução com mensagem de erro.

2.2 Resultados (situação 1 – ascendente)

Registros	Criação: transf.	Criação: comp.	Criação: tempo (ms)	Pesquisa: transf. (média)	Pesquisa: comp. (média)	Pesquisa: tempo (µs, média)
100	1	0	0,100	100	51,05	82,2
200	2	0	0,021	100	52,05	16,6
2.000	20	0	0,064	100	61,50	25,0
20.000	200	0	0,873	100	151,50	42,3
200.000	2.000	0	3,640	100	1.051,50	35,4
2.000.000	20.000	0	2.371,288	100	10.051,50	226,9

2.3 Análise

- **Criação – transferências:** exatamente n/100 (uma leitura por página), crescimento linear com fator pequeno.
- **Criação – comparações:** zero, pois o índice só copia a primeira chave de cada página.
- **Criação – tempo:** abaixo de 4 ms até 200.000 registros. Em 2.000.000 o arquivo tem cerca de 10 GB e não cabe no cache do sistema, então cada salto exige leitura real do disco e o tempo sobe para 2,4 s.
- **Pesquisa – transferências:** constante e igual a 100 em todos os tamanhos, porque a página inteira é trazida para a memória.
- **Pesquisa – comparações:** em média (n/100)/2 na tabela mais cerca de 50 na página, ou seja, crescem linearmente: de 51 (100 registros) para 10.051 (2.000.000). Uma pesquisa binária na tabela reduziria esse termo para $\log_2(n/100)$.
- **Pesquisa – tempo:** dominado pela leitura de uma página de cerca de 500 KB, por isso fica entre 17 e 82 µs até 200.000; em 2.000.000 sobe para 227 µs porque a página tem de vir do disco. O valor de 100 registros (82 µs), maior que o de 200, reflete a variação natural de medições na casa dos microssegundos.

3. Método 2 – Árvore binária de pesquisa em memória externa

A árvore binária é o único método cujo desempenho depende totalmente da ordem do arquivo. Com arquivo aleatório, a pesquisa custa cerca de 27 leituras mesmo com 2.000.000 de registros; com arquivo ordenado, a árvore vira uma lista e cada pesquisa lê em média n/2 nodos.

3.1 Implementação

- A árvore fica inteira em disco, no arquivo `arvorebinaria_<quantidade>_<situação>.bin`. Cada nodo guarda o registro completo e as posições (índice no arquivo) dos filhos esquerdo e direito, com -1 indicando filho vazio.
- **Criação:** cada registro do arquivo de dados é lido (1 transferência) e inserido descendo a partir da raiz; cada nodo visitado é lido do arquivo da árvore (1 transferência). O novo nodo é gravado no fim do arquivo e o ponteiro do pai é atualizado no lugar.
- **Pesquisa:** desce a partir da raiz lendo um nodo por vez do arquivo, até achar a chave ou chegar a um filho vazio.
- É o único método em que tanto a criação quanto a pesquisa usam somente memória externa.

3.2 Resultados

Registros	Situação	Criação: transf.	Criação: comp.	Criação: tempo (ms)	Pesquisa: transf. (média)	Pesquisa: comp. (média)	Pesquisa: tempo (µs, média)
100	Ascendente	5.050	9.900	11,17	50,05	99,10	75,0
100	Descendente	5.050	4.950	6,25	50,95	100,90	46,5
100	Aleatória	811	975	1,63	8,70	16,40	13,2
200	Ascendente	20.100	39.800	14,78	100,05	199,10	68,7
200	Descendente	20.100	19.900	14,68	100,95	200,90	69,5
200	Aleatória	1.793	2.294	2,64	9,25	17,50	17,4
2.000	Ascendente	2.001.000	3.998.000	1.396,20	1.000,05	1.999,10	634,2
2.000	Descendente	2.001.000	1.999.000	1.414,97	1.000,95	2.000,90	637,4
2.000	Aleatória	26.070	37.152	33,18	12,80	24,60	56,2
20.000	Ascendente	200.010.000	399.980.000	159.494,68	10.000,05	19.999,10	7.417,2
20.000	Descendente	200.010.000	199.990.000	163.856,85	10.000,95	20.000,90	7.509,4
20.000	Aleatória	374.285	506.246	475,72	17,60	34,20	77,6
200.000	Aleatória	4.757.935	6.828.679	5.641,92	21,00	41,00	32,4
2.000.000	Aleatória	53.193.586	76.939.985	817.895,59	27,40	53,80	4.760,3

Situações 1 e 2 com 200.000 e 2.000.000 registros não foram executadas (seção 1.5).

3.3 Análise

- **Criação – transferências:** nos arquivos ordenados são exatamente n(n+1)/2 (n leituras de dados mais 0 + 1 + … + (n−1) nodos visitados), crescimento quadrático: 5.050 para 100 registros e 200 milhões para 20.000. No aleatório são cerca de n mais n vezes a profundidade média; com 2.000.000 dá cerca de 26 nodos lidos por inserção, crescimento n·log n.
- **Criação – comparações:** no ascendente são n(n−1), o dobro do descendente (n(n−1)/2), porque descer à direita exige dois testes (menor que e maior que) e descer à esquerda só um. No aleatório crescem como n·log n (cerca de 38 por registro com 2.000.000).
- **Criação – tempo:** acompanha as transferências. Nos ordenados passa de 1,4 s (2.000) para cerca de 160 s (20.000), ou seja, 100 vezes mais para 10 vezes mais registros. No aleatório fica em 5,6 s com 200.000, mas chega a 818 s (13,6 min) com 2.000.000, porque o arquivo da árvore (cerca de 10 GB) não cabe na RAM e cada visita a um nodo vira um acesso aleatório ao disco.
- **Pesquisa – transferências:** ordenados, cerca de n/2 em média (as 20 chaves cobrem a lista de ponta a ponta); aleatório, logarítmicas: de 8,7 (100) para 27,4 (2.000.000), perto de $1{,}39 \cdot \log_2 n$, valor esperado para árvore binária aleatória.
- **Pesquisa – comparações:** cerca de duas por nodo visitado (igualdade e menor que), logo o dobro das transferências: 53,8 com 2.000.000 no aleatório e 20.000 com 20.000 nos ordenados.
- **Pesquisa – tempo:** cada nodo lido é um `fseek` mais um `fread` de cerca de 5 KB. No aleatório fica abaixo de 80 µs até 200.000 e sobe para 4,8 ms com 2.000.000 (leituras reais de disco); nos ordenados chega a 7,5 ms com apenas 20.000 registros.

4. Método 3 – Árvore B

A árvore B tem custo de pesquisa logarítmico e praticamente igual nas três situações: cerca de 34 comparações e nenhuma transferência com 200.000 registros.

4.1 Implementação

- Árvore B de ordem 2 (`ORDEM`), com até 4 registros e 5 filhos por página, mantida inteiramente em memória principal.
- **Criação:** cada registro do arquivo é lido uma vez (1 transferência) e inserido recursivamente; a busca da posição na página é sequencial e, em caso de overflow, a página é dividida e o registro do meio sobe para o pai. A árvore cresce pela raiz.
- **Pesquisa:** desce a partir da raiz fazendo pesquisa binária dentro de cada página.

- Como as páginas guardam os registros completos, a pesquisa não lê nada do disco: as transferências de pesquisa são 0 em todos os testes.

4.2 Resultados

Registros	Situação	Criação: transf.	Criação: comp.	Criação: tempo (ms)	Pesquisa: comp. (média)	Pesquisa: tempo (µs, média)
100	Ascendente	100	1.326	0,362	11,40	0,20
100	Descendente	100	1.342	0,271	11,00	0,20
100	Aleatória	100	1.270	0,216	11,90	0,15
200	Ascendente	200	3.144	0,387	12,90	0,20
200	Descendente	200	3.066	0,473	12,80	0,20
200	Aleatória	200	2.930	0,410	13,80	0,15
2.000	Ascendente	2.000	48.181	3,996	20,10	1,10
2.000	Descendente	2.000	43.493	4,667	20,70	1,00
2.000	Aleatória	2.000	42.139	4,301	21,20	0,30
20.000	Ascendente	20.000	650.318	90,067	28,00	6,30
20.000	Descendente	20.000	561.055	71,857	24,60	7,65
20.000	Aleatória	20.000	547.802	73,224	26,30	1,05
200.000	Ascendente	200.000	8.165.386	745,157	34,40	9,45
200.000	Descendente	200.000	6.879.919	703,516	34,40	13,70
200.000	Aleatória	200.000	6.745.805	750,088	34,30	2,20

Transferências na pesquisa: 0 em todas as linhas.

4.3 Análise

- **Criação – transferências:** exatamente n, pois o arquivo é lido uma única vez, sequencialmente.
- **Criação – comparações:** crescem como n·log n; por registro inserido são cerca de 13 (100 registros) e 34 a 41 (200.000). O arquivo ascendente é o mais caro (8,2 milhões contra 6,7 milhões no aleatório), porque toda inserção vai para a página mais à direita e a busca sequencial percorre todas as chaves de cada página; no descendente a posição é achada logo na primeira chave.
- **Criação – tempo:** cresce de forma quase linear com n (0,2 ms para 100, cerca de 0,75 s para 200.000) e varia pouco com a ordem do arquivo; o custo é dominado pela leitura e cópia dos registros de 5 KB.

- **Pesquisa – comparações:** logarítmicas e quase independentes da ordem do arquivo: de 11 (100 registros) para 34 (200.000). Multiplicar n por 10 acrescenta cerca de 7 comparações, o esperado para a altura de uma árvore balanceada.
- **Pesquisa – tempo:** abaixo de 14 µs em todos os casos. Com 20.000 e 200.000 registros o arquivo aleatório pesquisa mais rápido (1 a 2 µs) que os ordenados (6 a 14 µs), mesmo com o mesmo número de comparações; a diferença provável está no arranjo das páginas na memória (uso de cache do processador), não no algoritmo.
- **Memória:** com 200.000 registros o processo chegou a 2,2 GB de RAM (medido com `/usr/bin/time -l`), porque cada página reserva espaço para 4 registros de 5 KB.

5. Método 4 – Árvore B*

A árvore B* foi o método mais rápido em criação e em pesquisa: com 200.000 registros cria o índice em cerca de 0,33 s e pesquisa com 24 comparações, contra 0,75 s e 34 comparações da árvore B.

5.1 Implementação

- Os registros ficam apenas nas folhas (até 4 por folha, `MAX_REGISTROS_FOLHA`), encadeadas pelo ponteiro `proximo`; as páginas internas guardam só chaves (até 4) e 5 filhos (`MAX_CHAVES_INTERNA`). A árvore inteira fica em memória principal.
- A página é um registro variante (`union`); as páginas internas são alocadas só com o tamanho da parte interna (cerca de 64 bytes, em vez dos 20 KB de uma folha), o que reduz bastante o uso de memória.
- **Criação:** cada registro do arquivo é lido uma vez e inserido na folha correta. Folha cheia é dividida ao meio e a primeira chave da nova folha sobe como separadora; página interna cheia junta as 5 chaves num vetor temporário, divide ao meio e promove a chave central.
- **Pesquisa:** pesquisa sequencial nas páginas internas até chegar a uma folha e pesquisa sequencial na folha. Assim como na árvore B, não há leitura de disco na pesquisa (0 transferências).

5.2 Resultados

Registros	Situação	Criação: transf.	Criação: comp.	Criação: tempo (ms)	Pesquisa: comp. (média)	Pesquisa: tempo (µs, média)
100	Ascendente	100	1.219	0,217	9,80	0,20
100	Descendente	100	920	0,178	9,40	0,05
100	Aleatória	100	1.012	0,149	9,80	0,15

Registros	Situação	Criação: transf.	Criação: comp.	Criação: tempo (ms)	Pesquisa: comp. (média)	Pesquisa: tempo (µs, média)
200	Ascendente	200	2.819	0,289	10,75	0,15
200	Descendente	200	2.033	0,299	10,55	0,15
200	Aleatória	200	2.294	0,268	11,10	0,15
2.000	Ascendente	2.000	41.033	2,740	14,95	0,45
2.000	Descendente	2.000	24.968	2,718	14,80	0,30
2.000	Aleatória	2.000	31.107	2,668	15,65	0,25
20.000	Ascendente	20.000	536.071	28,870	19,45	1,35
20.000	Descendente	20.000	291.777	31,668	19,05	1,60
20.000	Aleatória	20.000	385.275	36,988	19,85	0,60
200.000	Ascendente	200.000	6.633.320	328,797	23,80	2,60
200.000	Descendente	200.000	3.331.780	310,087	23,35	3,65
200.000	Aleatória	200.000	4.604.473	390,476	23,80	0,85

Transferências na pesquisa: 0 em todas as linhas.

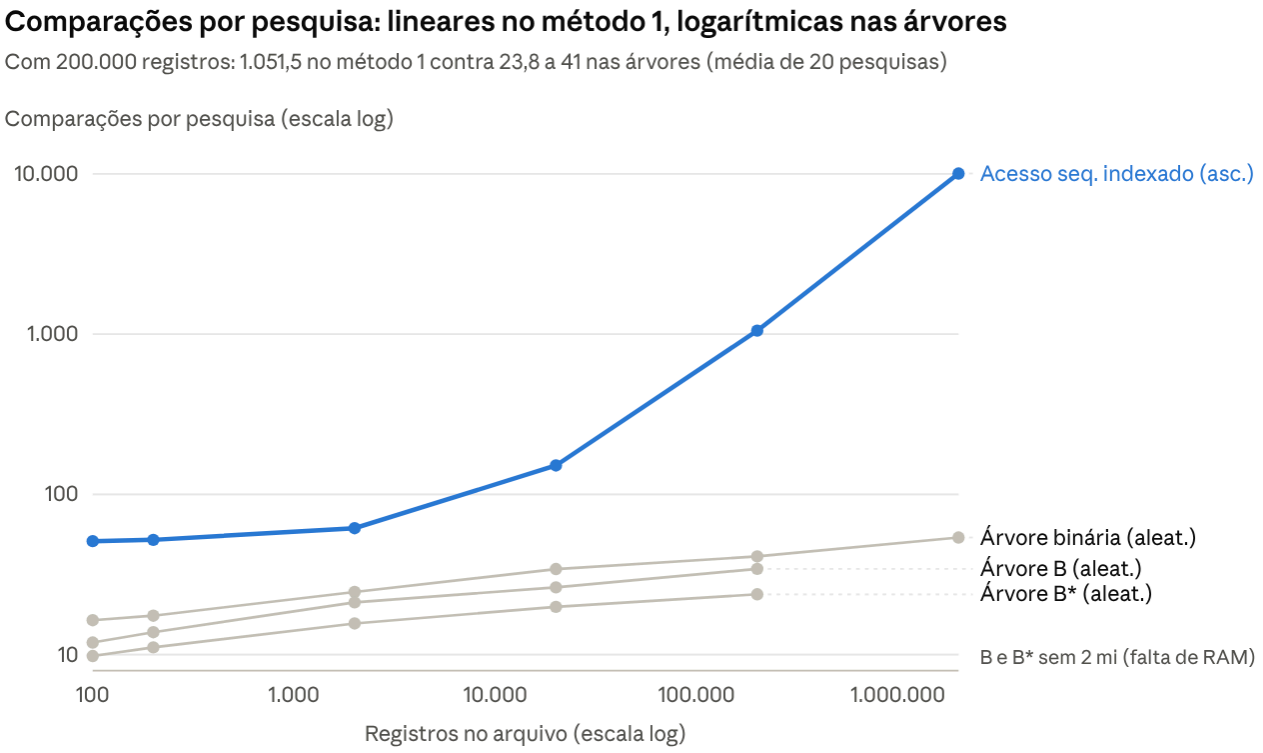
5.3 Análise

- **Criação – transferências:** exatamente n, uma leitura sequencial do arquivo.
- **Criação – comparações:** crescem como n·log n e dependem mais da ordem do arquivo que na árvore B. Com 200.000 registros o ascendente custa 6,6 milhões, o aleatório 4,6 milhões e o descendente 3,3 milhões: no ascendente a busca sequencial na folha e nas páginas internas percorre todas as chaves até o fim; no descendente para na primeira.
- **Criação – tempo:** quase linear em n e cerca de metade do tempo da árvore B (310 a 390 ms contra 700 a 750 ms com 200.000), porque as divisões de páginas internas movem apenas chaves inteiras, e não registros de 5 KB.
- **Pesquisa – comparações:** logarítmicas e praticamente iguais nas três situações: de 9,8 (100) para 23,8 (200.000). São menos que na árvore B porque cada chave de página interna custa uma comparação, enquanto cada passo da pesquisa binária da árvore B conta duas (igualdade e menor que).
- **Pesquisa – tempo:** até 3,65 µs, o menor entre os métodos. Como na árvore B, o arquivo aleatório tende a pesquisar mais rápido nos tamanhos maiores, efeito atribuível à disposição das páginas na memória.
- **Memória:** pico de 1,5 a 1,6 GB com 200.000 registros, contra 2,2 GB da árvore B, graças às páginas internas pequenas.

6. Conclusão

As árvores B* e B foram os métodos mais eficientes na pesquisa; o acesso sequencial indexado teve o índice mais barato; e a árvore binária só é viável com arquivo aleatório, situação em que foi o único método testado com 2.000.000 de registros guardando os dados somente em disco.

6.1 Análise comparativa



Experimentos do projeto (modo -E), 7/10/2026 · média de 20 chaves por ponto

Método	Situações aplicáveis	Criação (transferências)	Pesquisa (transferências)	Pesquisa (comparações)	Memória interna
1 – Acesso sequencial indexado	Só ascendente	n/100	100 (constante)	Linear em n	Só a tabela de páginas
2 – Árvore binária externa	Todas; viável só na aleatória	n·log n (aleat.) / n²/2 (ordenada)	log n (aleat.) / n/2 (ordenada)	Igual às transferências × 2	Nenhuma (tudo em disco)
3 – Árvore B	Todas	n	0	log n	Árvore inteira (2,2 GB com 200.000)
4 – Árvore B*	Todas	n	0	log n (a menor)	Árvore inteira (1,6 GB com 200.000)

- **Criação do índice:** o acesso sequencial indexado é imbatível (2.000 leituras e 3,6 ms com 200.000 registros) porque lê só um registro por página. Entre as árvores, a B* criou o índice em cerca de metade do tempo da B, e a binária aleatória foi a mais lenta (5,6 s com 200.000) por ler cada nodo visitado do disco.
- **Pesquisa – transferências:** as árvores B e B* não fazem nenhuma, pois mantêm os registros em RAM; o método 1 faz sempre 100; a binária aleatória faz cerca de $1{,}39 \cdot \log_2 n$ (27 com 2.000.000).
- **Pesquisa – comparações:** o gráfico mostra a diferença de crescimento. O método 1 é linear (de 200.000 para 2.000.000 as comparações passam de 1.051 para 10.051, quase 10 vezes mais); as três árvores são logarítmicas, e a B* fica sempre abaixo da B, que fica abaixo da binária.
- **Pesquisa – tempo:** B* (até 3,7 µs) < B (até 14 µs) < método 1 (17 a 227 µs) < binária aleatória (13 µs a 4,8 ms). A binária com arquivo ordenado é a pior de todas (7,5 ms com só 20.000 registros).
- **Ordem do arquivo:** não altera a pesquisa nas árvores B e B*, que se mantêm balanceadas, e altera apenas o custo de criação (o ascendente é o mais caro em comparações). Na árvore binária, arquivo ordenado transforma custo logarítmico em linear na pesquisa e em quadrático na criação.
- **Escala:** com 2.000.000 de registros, os métodos que dependem do disco ficaram muito mais lentos (criação de 2,4 s no método 1 e de 818 s na binária), e as árvores B e B* não puderam ser executadas por falta de memória.

6.2 Principais dificuldades de implementação

- **Árvore B*:** foi o método mais trabalhoso. A divisão de páginas internas precisou ser refeita (juntar as 5 chaves num vetor temporário antes de dividir), e os nomes de funções e tipos entraram em conflito com os da árvore B no mesmo programa e precisaram ser renomeados.
- **Uso de memória:** como as páginas das árvores B e B* guardam registros de 5 KB, o consumo de RAM cresce muito rápido. Na B* as páginas internas passaram a ser alocadas só com o tamanho da parte interna da `union`; mesmo assim não foi possível testar 2.000.000 de registros nas duas árvores. Guardar nas páginas apenas a chave e a posição do registro no arquivo resolveria isso, ao custo de uma transferência por pesquisa.
- **Árvore binária em disco:** manter os ponteiros como posições no arquivo e regravar o pai a cada inserção exigiu cuidado com `fseek`; além disso, a degeneração com arquivos ordenados tornou inviáveis os testes grandes nessas situações.
- **Medição justa:** foi preciso separar claramente o que entra em cada medição: a geração do arquivo e as impressões ficam fora, e toda comparação entre chaves passa pela macro `COMP`, para que a contagem seja uniforme entre os métodos.

- **Arquivos grandes:** um arquivo de 2.000.000 de registros ocupa cerca de 10 GB (mais 10 GB do arquivo da árvore binária), o que exigiu gerar os registros em lotes e apagar os arquivos entre os testes.